Premier League Model Upgrade Guide 2: Upgrades 7-18

Oct 12, 2026 · @Kyan

Can you do all 10?

Yes. Everything runs on what you already have (PostgreSQL, Excel, Python in VS Code) plus free data. Allow about 25–35 hours in total. Do them in this order, because the later ones are tested on the history that Upgrade 7 loads.

#	Upgrade (from my list)	Time	Needs first	What you'll learn
7	Load 2023-24 and 2024-25 (#1)	4–5 h	—	2024-25 becomes a second test season; Dixon–Coles gets history
8	Score season forecasts (#2)	1 h	7	How good your title / top-4 / relegation % really are
9	Ensemble with Dixon–Coles (#3)	1 h	7	Whether averaging models beats xg_50
10	Market-implied ratings (#4)	2 h	7, 9	The best single gain I found in testing
11	Rating uncertainty in sims (#5)	2 h	8	More honest season odds
12	Pi-ratings and xG Dixon–Coles (#6)	1.5 h	7	Two more benchmarks
13	Squad value prior (#7)	3 h	7	Better pre-season and promoted-club ratings
14	FPL availability + adjustments log (#8)	3 h + weekly	—	Whether team news helps next-week predictions
15	Player and match markets, bet builder style (new)	35–45 h	12–14	Player goals, assists, shots, SoT, fouls, fouled, tackles, saves, cards (1+, 2+, 3+); team and match corners, fouls, cards, booking points, red cards; slip checker

#	Upgrade (from my list)	Time	Needs first	What you'll learn
16	Take the password out of the code	30 min	—	A safe setup to automate and share
17	Automate the weekly pipeline (#9)	3–4 h	9–16	One command for the whole weekly run, player forecasts included
18	Portfolio packaging (#10)	4–6 h	all	GitHub repo, write-up, dashboard screenshots

What I tested before writing this. I rebuilt your 2025-26 backtest outside the database (it reproduces your xg_50 0.2120 and blend_15 0.2156 exactly) and ran Upgrades 9–12 on it. The expected numbers below come from that. They use only 2025-26, so your results after Upgrade 7 will differ a little, and that's the point of loading more seasons.

Model (2025-26, GW5–34, 1,440 matches)	RPS
Market + xg_50 ensemble (Upgrade 10)	0.2110
Market-implied ratings alone	0.2117
xg_50 (your current model)	0.2118
Dixon–Coles on xG (Upgrade 12)	0.2121
xg_50 + 25% Dixon–Coles (Upgrade 9)	0.2122
Dixon–Coles (goals)	0.2166
Pi-ratings, no history yet (Upgrade 12)	0.2283
Bookmakers, same matches	about 0.2061

New files (all in this chat; Python goes in your DC upgrade folder, SQL in your sql folder):

File	Upgrade
make_weekly_csvs.py (updated: adds `results` and a season argument)	7
dc_model.py (updated: `HISTORY_SEASONS = 2`, `RIDGE = 10`, password can come from an environment variable)	7, 16
23_upgrade7_history.sql	7

File	Upgrade
24_upgrade8_season_scores.sql	8
25_upgrade9_10_ensembles.sql, market_model.py	9, 10
dynamic_sims.py, benchmarks.py, 26_upgrade11_12_sims_benchmarks.sql	11, 12
27_upgrade13_squad_value.sql	13
fpl_availability.py, 28_upgrade14_availability.sql	14
weekly.py	15

Before you start: copy your new dc_model.py over the old one, then re-type your password on line 25 (with `%40` for the `@`). Back up the database once: pgAdmin → right-click prem_sims → Backup… → Format Custom → save as `prem_sims_before_upgrade7.backup`.

Upgrade 7: load 2023-24 and 2024-25

Why: every test so far rests on one season (1,790 predictions), and Dixon–Coles lost mainly because it had only one season to learn from. Two more seasons give it history and give you a second test season, 2024-25, with ten backtest runs built straight from the database. No Excel work is needed.

7.1 Download four files into a new folder `C:\Users\<you>\OneDrive\Prem Project\history`:

File to save	Where from
E0_2324.csv	`https://www.football-data.co.uk/mmz4281/2324/E0.csv` (opens or downloads; Save As)
E0_2425.csv	`https://www.football-data.co.uk/mmz4281/2425/E0.csv`
understat_2023-24.json	`https://understat.com/league/EPL/2023` → F12 → Network → F5 → filter `getLeagueData` → Response → copy into Notepad → save (UTF-8, All files)
understat_2024-25.json	the same with `league/EPL/2024`

7.2 Make the six CSVs (results, xg and odds for each season). Copy `make_weekly_csvs.py` into the history folder, open the folder in VS Code and run these six lines in the terminal:

```
python make_weekly_csvs.py results E0_2324.csv 2023-24
python make_weekly_csvs.py results E0_2425.csv 2024-25
python make_weekly_csvs.py xg understat_2023-24.json 2023-24
python make_weekly_csvs.py xg understat_2024-25.json 2024-25
python make_weekly_csvs.py odds E0_2324.csv 2023-24
python make_weekly_csvs.py odds E0_2425.csv 2024-25
```

Because each line ends with a season, the files are named after it: results_2023-24.csv, xg_2023-24.csv, odds_2023-24.csv and the same for 2024-25. No renaming needed. Each should report 380 rows (results also say `highest round_no 38`). Round numbers for past seasons are each team's game number, so `after GW10` means nobody had played more than 10 games.

7.3 Load them (pgAdmin, script 23). Open `23_upgrade7_history.sql` and run block by block:

Block	Does	Expect
7.3a	Adds the two seasons	INSERT 0 2 (or 0 1)
7.3b	Empties staging; then import results_2023-24 and results_2024-25 into stg_results, the two xg files into stg_xg, the two odds files into stg_odds	6 "completed" pop-ups
7.3c	Counts	760 / 760 / 760
7.3d	Unknown names	Probably Luton and Sheffield United, maybe Leicester, Southampton
7.3e	Lists your teams	If the club is there under a longer name (e.g. Luton Town) add only an alias; if not, add the team then the alias (templates in the script). Rerun 7.3d until it's empty
7.3f	Inserts the matches	INSERT 0 760
7.3g	2024-25 rebuilt vs your stored final table	0 rows
7.3h	2023-24 final table	INSERT 0 20

Block	Does	Expect
7.3i / 7.3j	xG and bookmaker odds	INSERT 0 760 each
7.3k	Everything loaded	380 / 380 / 380 for both seasons

7.4 Build the 2024-25 backtest runs (same script). These copy your Excel method exactly: in-season ratings are 25%-reverted goals per game relative to the league average; pre-season ratings come from the 2023-24 table, and the three promoted clubs from your Championship regression plus 25% reversion.

Block	Expect
7.4a: ten run labels (preseason, GW3 … GW34)	INSERT 0 10
7.4b: in-season ratings	INSERT 0 180
7.4c: pre-season ratings	INSERT 0 20
7.4d: check	10 rows, 20 / 20
7.4e: promoted clubs	Leicester, Ipswich, Southampton near the bottom of attack

These runs have ratings but no Excel simulations, which is fine: match predictions for every variant appear in v_scores automatically, and Upgrade 11 simulates them in Python.

7.5 Refit Dixon–Coles with history. Your new dc_model.py uses two previous seasons (`HISTORY_SEASONS = 2`). Set `ONLY_RUN = None`, save, run `python dc_model.py`. Expect 22 lines: 2024-25 runs fitted on 380+ matches, 2025-26 runs on 760+, and no more "skipped" for 2025-26 preseason and GW3.

7.6 Results across both seasons. Run the last block of script 23. Write the four models' RPS for 2024-25, 2025-26 and both into the dashboard's Upgrade log. If xg_50 still wins in 2024-25, Upgrades 2 and 5 are confirmed on a season they were never tuned on, which is a strong portfolio point.

Dashboard: Refresh All. 2024-25 runs now appear in Model comparison's C4 list (add them to Lists column A) and in Data_scores.

Upgrade 8: score the season forecasts

Why: your headline outputs are "Man City 59% to win the league" and "Coventry 76% to go down", but RPS only scores single matches. This scores the season forecasts themselves against the final table, so you can say how good they are and whether later upgrades improve them.

How it's scored (lower is better in every column):

- **Title, top-4 and relegation Brier:** for each team, (forecast chance − what happened)², averaged over the 20 teams. 70% for a team that finished top-4 scores 0.09; 70% for one that didn't scores 0.49.
- **Position RPS:** the same idea as your match RPS, over all 20 finishing positions, so finishing 5th when you said "most likely 4th" costs little and finishing 15th costs a lot.
- **Places out:** average gap between predicted and actual finish.

Steps (pgAdmin, script 24):

1. **8.1** creates two empty tables that Upgrade 11 will fill. Expect CREATE TABLE twice.
2. **8.2** and **8.3** create the views `v_season_scores` and `v_position_rps`. Expect CREATE VIEW. They read your Excel sims (v_sim_summary, v_position_probs), Dixon–Coles (dc_sim_summary) and Upgrade 11's sims, and score every run whose season has a final table (2024-25 and 2025-26).
3. **8.4** lists your Excel model run by run for 2025-26. Scores should fall steadily through the season (position RPS roughly 0.15 pre-season to 0.04 after GW34); a run that breaks the pattern is worth a look.
4. **8.5** compares your Excel model with Dixon–Coles on the same runs.
5. **8.6** is a calibration table for top-4 calls: in each band (0–20%, 20–40% …) the average forecast should be close to how often it happened.

Dashboard (optional, 15 min): load `v_season_scores` (Data → Get Data → ODBC → prem_sims → v_season_scores → Load To… → Table → New worksheet, rename `Data_season_scores`, background refresh off). Insert a PivotChart (line): Axis = gameweek, Legend = source, Values = Average of top4_brier, filter season = 2025-26. Title: `Season forecast accuracy (lower = better)`.

Upgrade log row: "8. Season forecast scoring — Excel model 2025-26: title / top-4 / relegation Brier ___ / ___ / ___, position RPS ___."

Upgrade 9: ensemble with Dixon–Coles

Why: professional forecasts are usually averages of several models, because different models make different mistakes. An ensemble here is just the average of two models' home/draw/away chances.

What to expect: on 2025-26 alone it did **not** help (xg_50 0.2118, 25% Dixon–Coles 0.2122, 50% 0.2131), because Dixon–Coles had one season of history. With Upgrade 7's history Dixon–Coles improves, so the result may change. Either answer is a finding for your log.

Steps (pgAdmin, script 25):

1. **9.1** creates `v_main_predictions`: your main model's predictions for every run in one place. Before 2026-27 GW6 it uses the xg_50 rebuild; from GW6 your exported ratings (A, J, K) already include blending and xG, so it uses them as they are (home_adv). This view is reused by Upgrades 10, 11, 15 and 17.
2. **9.2** checks it: 380 rows per run.
3. **9.3** writes two ensembles into `external_predictions`: `ens_dc_25` and `ens_dc_50`. Takes about a minute. Expect INSERT 0 (380 × the runs Dixon–Coles fitted, about 16,700 after Upgrade 7).
4. **9.4** compares them on 2025-26. Change `'2025-26'` to `'2024-25'` and run again for the second season.

Decision rule: adopt an ensemble only if it beats xg_50 in **both** seasons. If it does, run block 9.3 after every new run (Upgrade 17 does it for you).

Dashboard: in Model comparison, type `ens_dc_25` over `blend_5` in the model list (column K, rows 8–17) so it appears in the ranking and charts. The list has 10 slots; replace models you no longer need (blend_5, blend_10, promoted_reg).

Upgrade 10: market-implied ratings

Why: bookmakers still beat you, and Opta's forecasts use betting odds as an input. This model turns the odds of matches already played into team ratings: it finds the attack, defence and home-advantage numbers that best reproduce the bookmakers' win/draw/loss chances, then predicts the rest of the season with them. The odds were set before kick-off, so no results leak in.

What to expect (2025-26, GW5–34): the market model alone scores 0.2117, the same as xg_50 (0.2118), but the two disagree in useful ways: their 50/50 average scores **0.2110**, the best in my tests, closing about a seventh of the gap to the bookmakers (0.2061).

Steps:

1. Put `market_model.py` in the DC upgrade folder (it re-uses dc_model.py's password and code).
2. In VS Code: `python market_model.py`. Expect one line per run, e.g. `2025-26 after GW10: fitted on 480 matches with odds; home x1.2…`. Pre-season runs use the previous seasons' odds, so they need Upgrade 7.

3. In pgAdmin, script 25 block **10.3**: writes `ens_mkt_50` and `ens_3way` (main + market + Dixon–Coles). Expect INSERT 0 (380 × runs).
4. Block **10.4**: the full comparison against the bookmakers on the same matches. Repeat with `'2024-25'`.

Settings you can try (top of market_model.py, then rerun steps 2–4): `XI` = 0.003 (longer memory) or 0.008 (shorter); `HISTORY_SEASONS` = 1.

Decision rule: if `ens_mkt_50` beats xg_50 in both seasons, make it your headline match model: rerun steps 2–3 for every new run (Upgrade 17 automates it) and add `market` and `ens_mkt_50` to Model comparison's list. Your Excel season simulations stay as they are: this changes match predictions, not the sims.

Upgrade log row: "10. Market-implied ratings — market ___, ens_mkt_50 ___ vs xg_50 ___ (2025-26); 2024-25 ___ / ___ / ___."

Upgrade 11: rating uncertainty in the season simulations

Why: each of your 1,000 simulated seasons uses exactly the same ratings, as if they were known to be right. In reality a team rated 1.10 might really be 1.00 or 1.20, so your season odds come out a little too confident. FiveThirtyEight's club forecasts handled this; this upgrade does it simply: each simulated season first nudges every team's attack and defence by a random amount (spread sigma), shrinking as more games are played: sigma × √(15 / (15 + gameweek)).

What to expect (my 2025-26 rebuild, 10 runs): a modest, mixed improvement, so treat it as an experiment until Upgrade 7's second season agrees.

Sims	Position RPS	Top-4 Brier	Title Brier	Relegation Brier
Fixed ratings (dyn_0, like Excel)	0.0952	0.0920	0.0157	0.0269
Uncertainty 0.10 (dyn_10)	0.0938	0.0900	0.0160	0.0274

Steps:

1. Upgrade 8 must be done (it created the tables).
2. Put `dynamic_sims.py` in the DC upgrade folder and run `python dynamic_sims.py`. It simulates every run 10,000 times twice (sigma 0 and 0.10) using your main model's ratings, about 1–2 minutes per run. Expect lines like `2025-26 after GW10 dyn_10: leader Arsenal 54.0% title`.

3. pgAdmin, script 26 block **11.4**: both seasons, lower is better. **11.5** shows it run by run; **11.6** shows how the live 2026-27 odds change (expect the favourite a few points lower and long shots a few points higher).
4. To try other spreads, set `SIGMAS = [0.05, 0.15]` at the top, rerun step 2, and add `dyn_5`, `dyn_15` to the `IN (…)` list in 11.4.

If it helps in both seasons: add the same spread to Excel so the dashboard's season odds use it.

1. **Settings:** A39 `Rating uncertainty (Upgrade 11)`; A40 `Sigma`, B40 `0.10` (Formulas → Define Name → `Sigma`); A41 `Gameweek of this run`, B41 the run's gameweek, e.g. `7` (name it `RunGW`).
2. **Team Strength updated:** M1 `SIM ATT`, N1 `SIM DEF`. M2: `=J2*EXP(Sigma*SQRT(15/(15+RunGW))*NORM.S.INV(RAND()))`, N2: the same with `K2`. Fill down to row 21.
3. **Names:** Formulas → Name Manager → New: `SimAttack` = `='Team Strength updated'!$M$2:$M$21`, `SimDefence` = `='Team Strength updated'!$N$2:$N$21`.
4. **Fixture sims updated:** select columns F:G, Ctrl+H, find `Teams,Attack)` replace with `Teams,SimAttack)`, then find `Teams,Defence)` replace with `Teams,SimDefence)` → Replace All each time.
5. Press F9. Each of the 1,000 data-table trials recalculates the whole workbook, so every simulated season gets its own draw per team, exactly like the Python version. Attack and Defence (J:K) are untouched, so H2H and your ratings export don't change.
6. Set B40 to `0` to switch it off.

Upgrade 12: pi-ratings and Dixon–Coles on xG

Why: two more benchmarks that write into the same table as Dixon–Coles and are scored automatically.

- **Pi-ratings** (Constantinou & Fenton, 2013) are a football version of Elo: separate home and away ratings, updated after every match by the goal margin, with big wins counting less than proportionally. They need history to settle, so they suit Upgrade 7.
- **Dixon–Coles on xG** fits the same model as dc_model.py but on expected goals instead of goals, so it learns from chances rather than finishing luck.

I wrote both in plain Python (numpy/scipy) instead of the penaltyblog library, because penaltyblog wouldn't install on a recent Python here and may not on your Python 3.14 either.

What to expect (2025-26 only, no earlier history): dc_xg 0.2121, almost level with xg_50 (0.2118) and well ahead of goals-based Dixon–Coles (0.2166). Pi-ratings 0.2283 without history, but 0.2025 on the 2026-27 pre-season run where it had a season behind it.

Steps:

1. Put `benchmarks.py` in the DC upgrade folder (it uses dc_model.py's password and code).
2. `python benchmarks.py`. Expect two lines per run (`pi_ratings: 380 predictions saved`, `dc_xg: …`), or "skipped" where there are fewer than 50 matches of history.
3. pgAdmin, script 26 blocks **12.3** (2025-26) and **12.4** (2024-25).

Decision rule: these are benchmarks. If dc_xg beats xg_50 in both seasons, try it in an ensemble: copy block 9.3 of script 25, change `'dixon_coles'` to `'dc_xg'` and the source name to `'ens_dcxg_' ||`.

Upgrade 13: squad market value as a pre-season prior

Why: your pre-season ratings are last season's goals pulled 25% to average, so they miss summer transfers and are weakest for promoted clubs. Squad market value captures both, and Transfermarkt valuations have beaten the FIFA ranking and Elo as predictors in published work (Peeters, 2018). The prior matters most pre-season and fades through blending as games are played.

13.1 Collect 80 numbers. For each season 2023-24, 2024-25, 2025-26 and 2026-27, open Transfermarkt's Premier League overview for that season (the season selector on the competition page) and copy each club's **total market value** in €m. Use the value nearest 1 September (after the window). If the page only shows today's values for a past season, use each club's "market value development" chart for that date. 80 numbers, about an hour.

13.2 Make squad_values.csv in Excel, three columns: `season, team, value_m`, one row per club per season, e.g. `2025-26,Man City,1270`. Use the names your database knows (the football-data style: Man City, Man United, Nott'm Forest, Spurs or Tottenham, Newcastle). Save As CSV UTF-8.

13.3 Load it (script 27): 13.3a creates the tables and empties staging → import squad_values.csv into stg_squad_values → 13.3b unknown names (0 rows) → 13.3c INSERT 0 80 and 20 per season.

13.4 Value-implied ratings. `v_value_ratings` turns value into ratings: for each season it regresses earlier seasons' end-of-season ratings on log value ("how many times the league-average squad"), so a season's prior never sees its own results. 13.4b should show the richest clubs with the highest attack (about 1.3) and lowest defence (about 0.75).

13.5 Let variants use it. 13.5a adds a `value_weight` column (0 for every existing variant). 13.5b replaces `v_variant_ratings` with a version that blends the value rating into the pre-season rating when value_weight > 0; everything else is unchanged. **13.5c must show the same RPS as before** (xg_50 0.2120, blend_15 0.2156 for 2025-26). If not, stop and send me the result.

13.6–13.7 Test it. Two variants on top of xg_50: `value_33` and `value_50`. 13.7 splits RPS into pre-season, GW3–10 and GW15+, for both test seasons. Adopt if pre-season and GW3–10 improve in both seasons; GW15+ should barely move.

13.8 If adopted, use it in Excel:

1. New sheet **Squad values**: A team (your model's names), B value €m, C value attack, D value defence. Fill C:D from 13.4b for the season (run it with `'2026-27'`).
2. Settings: A43 `Squad value weight`, B43 your best weight (0.33 or 0.5), name it `ValueWeight`.
3. **Team Strength** (the pre-season sheet): L1 `PRE ATT (with value)`, M1 `PRE DEF (with value)`. L2: `=(1-ValueWeight)*B2+ValueWeight*XLOOKUP($A2,'Squad values'!$A:$A,'Squad values'!$C:$C,B2)`; M2: the same with `C2` and column D. Fill to row 21. Your B:C formulas, including the promoted clubs' regression, stay as they are.
4. **Team Strength updated:** select D2:E21, Ctrl+H, replace `'Team Strength'!$B$2:$B$21` with `'Team Strength'!$L$2:$L$21`, then `'Team Strength'!$C$2:$C$21` with `'Team Strength'!$M$2:$M$21`.
5. F9. The prior now feeds D:E through blending, and J:K through xG as before.
6. Each summer and on 3 February, update the values (and 13.3) after the window closes.

Upgrade 14: player availability and an adjustments log

Why: your ratings assume full strength. Bookmakers react to team news, which is part of why they're ahead. This adjusts **next gameweek's** fixtures only for who is injured, suspended or doubtful, using the official Fantasy Premier League API (free, no login).

Honest limit: past injury lists can't be downloaded, so this can only be tested going forward. Each run adds about 10 test matches; judge it at the end of the season.

14.1 Run the script. Put `fpl_availability.py` in the DC upgrade folder and run, with your run label:

```
python fpl_availability.py "2026-27 after GW6"
```

It prints the five teams most affected and writes `availability.csv` plus a dated copy in a `snapshots` folder. For each team it measures how much of its regulars' quality (the 14 players with most minutes, weighted by minutes and by price relative to their position) is

unavailable: attackers and midfielders for attack; goalkeepers, defenders and half of midfield for defence. With BETA = 0.3, losing a third of the attacking quality lowers the team's attack by 10% for that gameweek.

14.2 Model: Availability sheet. New sheet **Availability**. Paste availability.csv at A1 (columns A–H). Add your own columns:

Column	Header	What you type
I	manual_att	1 unless you want to adjust (e.g. 0.95 after a manager is sacked)
J	manual_def	1 unless adjusting
K	reason	why, with the date: this is your adjustments log
L	final_att	`=D2*I2`
M	final_def	`=E2*J2`

Check column A uses your model's team names (the script converts Man City, Man Utd, Spurs, Nott'm Forest and Hull; fix any other by hand).

14.3 Settings: A45 `Use availability? (Yes/No)`, B45 `Yes` (name `UseAvail`); A46 `Next gameweek`, B46 e.g. `7` (name `NextGW`).

14.4 Fixture sims updated: helper columns N:Q (N home attack, O home defence, P away attack, Q away defence factor):

- N2: `=IF(AND(UseAvail="Yes",$A2=NextGW),XLOOKUP($D2,Availability!$A:$A,Availability!$L:$L,1),1)`
- O2: `=IF(AND(UseAvail="Yes",$A2=NextGW),XLOOKUP($D2,Availability!$A:$A,Availability!$M:$M,1),1)`
- P2: as N2 with `$E2`; Q2: as O2 with `$E2`. Fill N2:Q2 down to row 381.
- In F2's BINOM.INV part multiply the home attack by `N2` and the away defence by `Q2`: `(XLOOKUP(D2,Teams,Attack)*N2*XLOOKUP(E2,Teams,Defence)*Q2*Average*HomeAdv)/10000`. In G2: `(XLOOKUP(E2,Teams,Attack)*P2*XLOOKUP(D2,Teams,Defence)*O2*Average*AwayAdv)/10000`. Fill down.

Only next week's fixtures change, so the season sims carry this week's injuries for one gameweek and your exported ratings (J:K) are untouched.

14.5 Database (script 28): 14.5a once; each run 14.5b: TRUNCATE, import availability.csv into stg_availability, INSERT (expect INSERT 0 20).

14.6–14.7 The test. `v_availability_test` predicts each run's next-gameweek matches from your exported ratings with and without that run's factors. After 8–10 runs, 14.7 gives the verdict: keep it if "with availability" has the lower RPS. To try another BETA, change it in the script for future runs; the snapshots keep the old ones.

Weekly: run 14.1 on the day before the next gameweek (team news is freshest), paste into Availability, set NextGW, then F9. Upgrade 17 does the download and database steps for you.

Upgrade 15: player and match markets (bet builder style)

Why: your team model already predicts each side's expected goals in every match-up. This upgrade adds the markets that bet365, Paddy Power and Betfair put in their bet builders. On the player side that means goals, assists, shots, shots on target, fouls, being fouled, tackles, saves and cards, each at 1+, 2+ and 3+. On the team and match side it means corners, fouls and cards for each team, over/under lines for match corners, cards and booking points, and red cards. It also includes a slip checker that works out the chance of a whole bet builder landing. Every output is a probability, not odds. About 35–45 hours over several sessions. Do it before Upgrades 16–17 so the automation wraps the finished system once.

What the sites offer (checked October 2026). The markets below are the ones that appear across bet365, Paddy Power and Betfair bet builders:

- **Player markets.** Anytime and 2+ goals and hat-trick; assist; goal or assist; shots and shots on target at 1+, 2+ and 3+; fouls committed and fouls won at 1+ and 2+ (Paddy Power also has a combined foul-involvements market, and bet365 quotes fouls committed as over/under 0.5, 1.5 and 2.5); tackles at 1+ and 2+; to be booked; to be sent off.
- **Team and match markets.** Match corners over/under, team corners over/under, match and team cards over/under, booking points (10 for a yellow and 25 for a red, so a second yellow is 35), team fouls (not offered by every site), and a red card in the match or for a team.
- **How they settle.** Player fouls are settled on Opta data. A foul won excludes handballs, dives, back passes and similar free kicks. Card markets differ by site: some count a red as 2 cards, and a second yellow can count differently. The model therefore stores yellows and reds separately so any site's rule can be calculated, and you should always check the site's own rules page.

How it works. Each market is a count (goals, shots, corners, cards). For each one the model predicts an average, λ, for the player or team in that fixture, then turns it into a chance for each line:

```
\lambda_{player} = \text{team expected stat} \times \text{player share}
\times \frac{\text{minutes}}{90}, \qquad P(X \ge k) = 1 - \sum_{i=0}^{k-1}
\frac{e^{-\lambda}\lambda^{i}}{i!}
```

That is the Poisson formula. 1+ is 1 − e^−λ, 2+ is 1 − e^−λ(1 + λ), and 3+ also subtracts e^−λ·λ²/2. Corners, fouls and cards vary more from game to game than Poisson allows, so team markets use the negative binomial, a Poisson with one extra spread setting fitted from your history (step 15.4).

Worked example: Haaland has 4.76 xG of City's 11.0 team xG, a 43% share. If your model gives City 2.1 expected goals and he plays 85 minutes, λ = 2.1 × 0.43 × 85/90 ≈ 0.85. That gives a 57% chance he scores, 21% for 2+ and 5.5% for a hat-trick.

The markets and where the data comes from:

Market	Lines	How it's predicted	Free data	Status
To score	1+, 2+, 3+	team xG × player xG share × minutes	Understat player xG and minutes	Ready
To assist; goal or assist	1+	team xG × player xA share × minutes; goal or assist = 1 − e^−(λ goal + λ assist)	Understat xA	Ready
Shots	1+, 2+, 3+	team shots × player shot share × minutes	Understat shots; football-data team shots (HS/AS)	Ready
Shots on target	1+, 2+, 3+	player shots × his on-target rate	Understat shot results (goal + saved = on target); football-data HST/AST	Ready
Fouls committed	1+, 2+, 3+	fouls per 90 × minutes × opponent and referee factors	FPL-Core-Insights per-match player file (fouls_committed, was_fouled)	Ready

Market	Lines	How it's predicted	Free data	Status
Fouls won (to be fouled)	1+, 2+, 3+	fouls won per 90 × minutes × opponent's fouling rate	as above	Ready
Tackles	1+, 2+	tackles per 90 × minutes × opponent possession factor	FPL-Core-Insights (tackles, from 2024-25)	Ready
To be booked	1+	card rate per 90 × minutes × match and referee factors	FPL yellow and red cards per match	Ready
To be sent off	1+	team red rate × player's share of team cards × minutes	FPL red cards; football-data HR/AR	Very rare: show, test over a full season
Keeper saves	2+, 3+, 4+	opponent's expected shots on target − expected goals conceded	football-data HST/AST; FPL saves	Ready
Team corners; match corners O/U	team 3.5–6.5; match 8.5–12.5	corner ratings like your goal ratings	football-data HC/AC	Ready
Team fouls; match fouls O/U	team 9.5–12.5; match 20.5–24.5	foul ratings × referee factor	football-data HF/AF and Referee	Ready
Team cards; match cards O/U	team 0.5–2.5; match 2.5–5.5	card ratings × referee factor	football-data HY/AY, HR/AR, Referee	Ready
Booking points O/U	match 30.5–50.5	simulated yellows × 10 + reds × 25	as above	Ready
Red card in match / for a team	yes or no	team red rate × referee factor, heavily shrunk	football-data HR/AR	Very rare: as above

15.1 Data (about 1–2 h). One script, `player_data.py`, downloads every source and loads it into prem_sims, and `29_upgrade15_data.sql` checks it. Both are written and were tested on the real GitHub data. Football-data and the database steps can only run on your PC.

Source	Seasons	What it provides	How
FPL-Core-Insights (GitHub)	2024-25 → now	per-match player minutes, goals, assists, xG, xA, shots, shots on target, fouls committed, fouled, tackles, saves; team match stats	script, automatic
vaastav FPL archive (GitHub)	2022-23 → 2025-26	per-fixture minutes, starts, yellow and red cards, saves, xG, xA (tackles from 2025-26)	script, automatic
FPL-Core-Insights' copy of FPL's per-gameweek stats	2026-27	the same FPL fields for the current season	script, automatic
Understat (via the vaastav archive)	2023-24	season totals of shots, xG, xA for each player (the starting point for 2024-25)	script, automatic
football-data.co.uk	2021-22 → now	team corners, fouls, cards, shots, shots on target, referee	script, automatic
premierleague.com "Match officials for Matchweek N"	each gameweek	upcoming referees	you type them in (about 2 minutes)

The script creates six tables:

- `pl_players`: one row per player per season, with his club and position.
- `pl_player_matches`: per-match player stats.
- `pl_player_fpl`: per-fixture or per-gameweek FPL stats, including cards.
- `pl_player_understat`: 2023-24 season totals.
- `team_match_stats`: football-data and FPL-Core-Insights rows, marked by `source`.
- `fixture_referees`: upcoming referees.

Players are identified by FPL's player code, which stays the same across seasons.

Steps:

1. Put `player_data.py` in the DC upgrade folder (next to dc_model.py; it uses the same database connection) and `29_upgrade15_data.sql` with your other SQL files.
2. In the VS Code terminal run `python player_data.py history`. The first run takes about 5–10 minutes, and it prints a line for every source and season. Expect:
   - FPL-Core-Insights: 380 matches and about 11,500 player rows for 2024-25 and 2025-26, plus the games played so far in 2026-27.
   - FPL archive: about 11,000–11,600 rows per season.
   - Understat: about 95% of minutes matched to FPL players.
   - football-data: 380 matches per season, each with a referee; about 10 corners, 22 fouls and 4 yellows per match.

   Downloads are cached in `player_data_cache/`, and a CSV copy of each table goes into `player_data/` for Excel.
3. In pgAdmin, run block 15.1a, the team-name check. For any club listed (FPL spellings such as Sheffield Utd or Luton may be), add an alias using the example in the block, then rerun 15.1a until it returns 0 rows.
4. Run 15.1b, which creates `v_team_names` and `v_team_match_stats`. These put your team ids, match ids and booking points on the football-data rows.
5. Run checks 15.1c to 15.1h; each block says what to expect. The key ones:
   - 380 matches per past season, each with a referee.
   - From 2023-24, every football-data match links to your matches table.
   - The two sources agree on corners and cards almost exactly, and on fouls to within about half a foul per match.
   - Player goals equal the score in about 9 matches in 10 (own goals make up the rest), with 11 starters per team.
   - FPL cards are within about 5% of football-data's.
6. Run 15.1i to create `v_referee_stats`, then 15.1j to see the strictest and most lenient referees.

Weekly, from now on:

1. After the results are in, run `python player_data.py update`, which refreshes the current season only and takes about a minute.
2. Run `python player_data.py referees`. This writes `referees_next.csv` with the next gameweek's fixtures.
3. Fill in the referee column from the premierleague.com "Match officials for Matchweek N" page and save. Full names are fine: the script turns "Anthony Taylor" into "A Taylor", the spelling football-data uses.

4. Run `python player_data.py referees load`, then block 15.1k. Any referee without history is either new (factor 1) or spelled differently.

Data notes:

- **Scores and team stats:** a few scores in FPL-Core-Insights' match file are wrong, though its player rows are right. Use your own matches table and football-data for scores and team stats, and FPL-Core-Insights only as the cross-check.
- **Starts:** its start times are unreliable, so the script takes who started, and each player's club in every fixture, from FPL.
- **Licence:** FPL-Core-Insights has no licence file, so keep its data out of your public repository. Add `player_data/` and `player_data_cache/` to `.gitignore`; the script itself is fine to share.

15.2 Expected minutes (about 30 minutes). This drives every player market and is the weakest input, so it's built and tested on its own first. `player_minutes.py` predicts, for every player before every gameweek, his chance of starting, his chance of coming on, and his expected minutes. It's stored in the `player_minutes` table, along with a simple baseline to beat: his average minutes over the team's last 5 matches.

The method (tuned on 2024-25, confirmed on 2025-26). For each team before each gameweek:

1. **Squad:** everyone who played for the team in its last 30 league matches and is still at the club, so players back from injury are included.
2. **Chances of starting and coming on:** how often he started or came on in those matches, with each match counting 70% as much as the one after it. The last 5 matches carry about 83% of the weight.
3. **Next gameweek only:** both chances are multiplied by FPL's chance of playing, so injured or suspended players get 0.
4. **Scaling:** the team's start chances are scaled to add up to 11, with nobody above 97%.
5. **Minutes per appearance:** minutes when he starts is the average of his last 8 starts. Minutes off the bench is the average of his last 8 sub appearances. A player with neither uses his position's average.
6. **Expected minutes:** P(start) × start minutes + P(sub) × sub minutes.

A 5-match window with gentler weighting was clearly worse. Going to 30 matches cut the start Brier from 0.161 to 0.129 in 2024-25, and from 0.175 to 0.131 in 2025-26.

Steps:

1. Put `player_minutes.py` in the DC upgrade folder and run `python player_minutes.py backtest`. It takes about 2–4 minutes and prints one line per gameweek, each showing 11.0 expected starters per team: 37 gameweeks for 2024-25 and 2025-26, plus the

2026-27 gameweeks played so far.

2. In pgAdmin, run block 15.2a, the storage check. Expect 37 gameweeks for each full season, about 11 starters and about 985 minutes per team.
3. Run 15.2b, which creates `v_player_match_ids` and `v_minutes_test`. Expect CREATE VIEW twice.
4. Run 15.2c, the test. On the test data the model beat the baseline in both seasons:
   - **Minutes error:** 27.5 vs 28.6 in 2024-25, and 28.3 vs 30.0 in 2025-26.
   - **Start Brier:** 0.130 vs 0.140, and 0.136 vs 0.153.
   - **Bias:** close to 0 in both seasons.
5. Run 15.2d, the same test by position. The model should win in every row.
6. Run 15.2e, the calibration table. Bands from 10% to 90% should be within about 5 points.
   - **Top band:** forecasts of 90% or more start about 89% of the time. That's because the backtest has no injury news; the live run adds FPL's flags.
   - **Bottom band:** about 6% of the 0–10% band start, mostly players returning from long absences.
7. **Each week:** after entering the results, run `python player_minutes.py next` and check the likely line-ups with 15.2f (change the team name in the block).

Decision: keep it if the model beats the baseline on both measures in both seasons. If it does, 15.3 onwards uses these minutes.

Result (your database, 14 Oct 2026): adopted.

- **Minutes error (model vs baseline):** 28.8 vs 30.1 in 2024-25, and 27.7 vs 29.1 in 2025-26.
- **Start Brier (model vs baseline):** 0.141 vs 0.155, and 0.131 vs 0.145.
- **By position:** the model wins in all 8 position × season rows.
- **Bias:** about −0.1 minutes.
- **Calibration:** the middle bands are within about 5 points. Forecasts of 90% or more start 88.5% of the time, because the backtest has no injury news.

15.3 Player rates per 90 (about 30 minutes). `player_rates.py` gives every squad player, before every gameweek, a per-90 rate for nine stats: xG, xA, shots, shots on target, fouls committed, fouled, tackles, yellow cards and red cards. In 15.5 these become shares of the team's expected totals. They're stored in `player_rates`, and backtest rows also hold what the player actually did in the next match.

```
\text{rate per 90} = 90 \times \frac{\text{this season} + W \times \text{last season} + K \times \text{position average}}{\text{minutes this season} + W \times \text{minutes last season} + K}
```

- **W** is how much last season counts: 0.4 for every stat. Cards also count the season before that, at 0.4 × 0.4.
- **K** is how many minutes the position average is worth. It's 200 for xG, shots and fouled; 300 for xA, shots on target, fouls and tackles; and 600 for yellows. A regular soon moves to his own numbers, while a player with few minutes stays near his position's average.
- **Reds** use the position average only (K = 5000). A player's own red cards are too rare to mean anything.
- **How the settings were chosen:** on 2024-25, then confirmed on 2025-26. 2024-25 can't tell how much last season should count for shots on target, fouls, fouled and tackles, because there's no 2023-24 data for them. So one shared W = 0.4 is used, sitting between what xG, xA, shots and yellows chose (0.25–0.5).

The test uses the minutes each player actually played, so it judges the rates separately from 15.2. It scores each player's chance of 1 or more, and of 2 or more, in the next match. The baseline is his raw rate this season. For goals and assists the baseline uses actual goals and assists, which shows whether xG is a better guide than goals.

Steps:

1. Put `player_rates.py` in the DC upgrade folder and run `python player_rates.py backtest`. It takes under a minute. Expect `Saved about 39,000 player rows for 78 gameweeks`, with about 23,000 of them having played in the predicted match.
2. In pgAdmin, run 15.3a to check the storage. It should show the same gameweeks as 15.2a, with average xG per 90 about 0.15, fouls about 1.1 and yellows about 0.19.
3. Run 15.3b, the 1+ test, and 15.3c, the 2+ test. The model should have the lower Brier for every stat in both seasons. Predicted and actual totals should be close.
4. Run 15.3d for a sense check of who's top for xG (or any other stat).
5. **Each week:** after `python player_minutes.py next`, run `python player_rates.py next`.

Result on the test data: the model beat the baseline for all nine stats in both seasons, at 1+ and at 2+. Some examples (model vs baseline):

Stat	2024-25	2025-26
Goals (from xG)	0.0699 vs 0.0727	0.0673 vs 0.0728
Fouls	0.1995 vs 0.2051	0.2047 vs 0.2150
Yellows	0.1125 vs 0.1182	0.1068 vs 0.1142

The totals are close for every stat but two:

- **Goals:** xG predicts about 10% more goals than were scored.
- **Assists:** xA predicted 661 assists in 2025-26 against 914 actual.

These don't matter, because 15.5 scales each team's players to the team's expected goals and assists. The rates set each player's share; the team total comes from the match model.

Result (your database, 14 Oct 2026): adopted.

- **The test:** the model beat the baseline in all 18 rows of the 1+ test and all 10 rows of the 2+ test.
- **Examples** (model vs baseline, 1+ Brier):

Stat	2024-25	2025-26
Goals	0.0705 vs 0.0733	0.0665 vs 0.0703
Fouls	0.2020 vs 0.2073	0.2020 vs 0.2109
Yellows	0.1138 vs 0.1196	0.1057 vs 0.1109

- **2+ chances:** predicted within about 1–2 points of actual. For example, 2+ shots happened 21.9% of the time against 23.3% predicted.
- **Sense check:** Haaland tops xG per 90 (0.76).

15.4 Team counts: corners, fouls, cards, shots, shots on target, tackles (about 45 minutes, mostly waiting). `team_counts.py` predicts each team's count in every match for seven stats, built like your goal model. The results go in the `team_counts` table, one row per match × stat × side (home, away and match total).

- **League averages:** home and away counts per match, with recent matches weighted more by a half-life in days.
- **Team ratings:** a "for" rating (his count against the league average) and an "against" rating (what opponents get against him). Teams with few matches are pulled towards 1, by K matches' worth of average.
- **Expected count:** home = home average × home "for" × away "against", and the same for the away side.
- **Referee factor (fouls and cards only):** his matches compared with that season's average, pulled towards 1.
- **Game state:** × exp(± β × expected goal difference), because favourites win more corners and take more shots.
- **Spread:** the negative binomial setting r, fitted on the seasons before the one being predicted. It's needed for the over/under lines.
- **Baseline to beat:** the league average with home advantage only.

The script chooses the settings for each stat on 2024-25 and prints them. It then prints the 2025-26 check, saves the settings in `team_counts_settings.json`, and the weekly run uses them. The settings are the half-life (90–730 days), K (3–1000, where 1000 means ignoring team differences), β (−0.2 to +0.4) and the referee strength (none, 10 or 30 matches).

Fixed while building this, which affects 15.2 and 15.3 too:

- **Leak from rearranged fixtures:** a match brought forward could be in its own history. All three scripts now leave out the matches being predicted.
- **Cutoff:** it's now the day before the predicted round's first kick-off. Before, the latest date in earlier rounds was used, so a postponed game played months later could push it into the future.
- **2026-27 tackles:** FPL-Core-Insights' `tackles` column is empty this season, and `player_data.py` now uses `tackles_won`, which matches FPL's tackle count.

On the test data, 15.2 and 15.3 still beat their baselines everywhere after these fixes.

Rerun on your database after the fixes (15 Oct 2026): both still adopted. One more bug was found and fixed along the way. The number conversion had turned `was_home` into 1/0, so some players were matched to the reverse fixture. These are the final numbers (model vs baseline):

15.2, expected minutes:

Season	Minutes error	Start Brier
2024-25	29.1 vs 30.5	0.145 vs 0.158
2025-26	28.6 vs 30.3	0.139 vs 0.156

15.3, player rates: the model won all 18 rows of the 1+ test, with 23,153 player-matches tested. Some examples (1+ Brier):

Stat	2024-25	2025-26
Goals (from xG)	0.0707 vs 0.0756	0.0673 vs 0.0728
Fouls committed	0.2040 vs 0.2119	0.2047 vs 0.2151
Yellow card	0.1144 vs 0.1229	0.1068 vs 0.1142

Steps:

1. **Copy in the new scripts:** replace `player_data.py`, `player_minutes.py` and `player_rates.py`, and add `team_counts.py`, all in the same folder as `dc_model.py`.

2. **Rerun the earlier steps:**
   - `python player_data.py update`, which fixes 2026-27 tackles.
   - `python player_minutes.py backtest`, then `python player_rates.py backtest`.
   - In pgAdmin, rerun 15.2c and 15.3b, and check the model still beats the baseline.
3. **Run the team model:** `python team_counts.py backtest`. It takes about 15–30 minutes, while it tries 60–180 settings per stat. It prints the chosen settings, the 2024-25 and 2025-26 log scores against the baseline, and the rows saved.
4. **pgAdmin checks:**
   - 15.4a: storage.
   - 15.4b: log score, which should beat the baseline for every stat in both seasons. Reds may only tie.
   - 15.4c: the over/under Brier at the usual line, for team and match totals.
   - 15.4d: no bias.
   - 15.4e: whether the referee factor works. Strict referees should see more cards than the baseline expects.
5. **Each week:**
   a. `python player_data.py referees`, fill in the referees, then `python player_data.py referees load`.
   b. `python team_counts.py next`.
   c. Run 15.4f for the fixture table.

Result on the test data (made-up referees, so no referee factor was chosen):

- **Log score:** every stat except reds beat the baseline in both seasons. For example, corners 2.346 against 2.385 and fouls 2.585 against 2.610 in 2025-26.
- **Reds:** level with the baseline, because they barely differ between teams. Only the referee can help there.
- **Team over/under:** Brier beat the baseline for corners, fouls, shots, shots on target, tackles and yellows.
- **Match totals:** a few match lines only tie, because the sum of two teams is harder to call.

Your real referees decide whether a referee factor is kept.

Result (your database, 15 Oct 2026): adopted.

Settings chosen on 2024-25:

Stat	Half-life	K	Game-state	Referee
Corners	180 days	20	+0.2	none

Stat	Half-life	K	Game-state	Referee
Fouls	180 days	8	0	K 30
Yellows	365 days	8	0	none
Reds	—	1000 (league average)	0	none
Shots	90 days	3	0	none
Shots on target	180 days	8	0	none
Tackles	180 days	3	0	none

Log score (model vs baseline, lower is better):

Stat	2024-25	2025-26
Corners	2.398 vs 2.498	2.334 vs 2.381
Fouls	2.617 vs 2.694	2.572 vs 2.607
Shots	2.944 vs 3.098	2.866 vs 2.958
Shots on target	2.186 vs 2.265	2.128 vs 2.180
Tackles	3.008 vs 3.093	2.982 vs 3.056
Yellows	1.688 vs 1.717	1.617 vs 1.629
Reds	0.252 vs 0.252 (tie)	0.207 vs 0.207 (tie)

Over/under at the usual line:

- **Team lines:** beat the baseline for every stat except reds. Examples: team corners 0.227 vs 0.248, team shots 0.207 vs 0.244, team shots on target 0.217 vs 0.241.
- **Match totals:** won for fouls, shots, tackles and yellows. They tied for match corners, match shots on target and reds.

Bias: within about 3% for fouls, yellows, shots and shots on target. Corners ran 6% high in 2024-25 (10.9 expected vs 10.3 actual), 2% in 2025-26. Tackles ran 2–5% high.

Referee factor for fouls:

- **Lenient referees** (71 matches): baseline 21.9, model 20.9, actual 20.8.
- **Strict referees** (32 matches): baseline 22.0, model 23.0, actual 22.2.

So the factor points the right way and is accurate for lenient referees, but slightly overshoots for strict ones.

Yellows: they chose no referee factor, because the team ratings capture most of the difference. A weaker referee pull (K 60–100) is worth testing later.

Referee stats. A view, `v_referee_stats`, gives one row per referee per season, plus an all-seasons row. Columns:

- matches refereed
- yellow cards, red cards, booking points and fouls per game
- each of those as a ratio to that season's league average (the yellow and foul ratios, shrunk as above, are the referee factors the model uses)
- his home vs away split for cards and fouls, to see whether he books away sides more

It's built from football-data's Referee, HY/AY, HR/AR and HF/AF columns, 2021-22 onwards, so every referee's numbers cover up to five seasons (about 100 games for a regular), with recent seasons weighted more. football-data spells names as initial and surname (for example "A Taylor"), so `fixture_referees` must use the same spelling. Check: every referee in `fixture_referees` should match a name in the view; a new referee with no history gets a factor of 1. Bear in mind that fouls depend more on the teams than the referee, so expect the foul factors to sit closer to 1 than the card factors.

15.5 Attacking player markets (about 1 hour). `player_markets.py` gives every squad player in every fixture his chance of each attacking market: to score (1+, 2+, 3+), to assist, goal or assist, shots (1+, 2+, 3+) and shots on target (1+, 2+, 3+). The results go in the `player_markets` table.

- **Team totals:** each team's expected goals comes from your main model, adjusted to match your ensemble's (ens_dcxg_50) win, draw and loss chances, and lives in the new `match_lambdas` table (block 15.5a). Expected assists = expected goals × the league's assists per goal. Expected shots and shots on target come from 15.4. The opponent enters here: a weak defence raises the team's expected goals and shots, so it raises every player's chances.
- **His share:** his 15.3 rate × his 15.2 expected minutes, as a share of the team's total. The team's players then add up to the team's expected total, with about 3% of goals left for own goals.
- **How much of the team and opponent scaling to use (0–1):** chosen per stat on 2024-25 and checked on 2025-26, printed by the backtest and saved in `player_markets_settings.json`.
- **His chance:** a Poisson chance, mixing "he starts" and "he comes on", given that he plays. That's because bookmakers void player bets when the player doesn't play.
- **Two comparisons in the test:** the baseline, which is his raw rate × his usual minutes, and "rates only", which is his 15.3 rate with no team or opponent. Rates only shows what the opponent adds.

Steps:

1. **Expected goals table.** In pgAdmin, run block 15.5a: create `match_lambdas`, fill it, then run the check. Your match model only has saved runs for 9 gameweeks in each full season, so then run 15.5a-ii, which fills every other week from the latest earlier saved run (no leak), and rerun the check. Result: 35 gameweeks and 350 fixtures for 2024-25 and 2025-26, 1 for 2026-27; home about 1.5–1.6 and away about 1.2–1.3 expected goals. Only the saved weeks are adjusted to the ensemble's win chances; filled weeks use the main model's expected goals as they are.
2. **Backtest.** Put `player_markets.py` in the folder and run `python player_markets.py backtest`, which takes about a minute. It prints:
   - the league's assists per goal;
   - how many fixtures were matched to the ensemble;
   - the chosen scaling for goals, assists, shots and shots on target, with Brier for none and full in both seasons;
   - a check that players' expected goals per team average about 1.4.
3. **Storage and test.** Run 15.5b for the storage check, then 15.5c, the test. The model should beat the baseline in every row, and predicted percentages should be close to actual.
4. **Calibration and the opponent effect.** Run 15.5d, calibration for "to score". Then 15.5e: one player's scoring chance against every opponent in a season. Weaker defences, especially at home, should be at the top.
5. **Each week:**
   a. After the weekly run, rerun 15.5a so the new run gets expected goals.
   b. Run `python player_minutes.py next`, `python player_rates.py next`, `python team_counts.py next` and `python player_markets.py next`.
   c. Run 15.5f, the likeliest scorers.

Result on your data (adopted). Tested on 35 gameweeks a season (about 115,000 player-games each), after two fixes described below.

Market	Brier 2024-25 (model / baseline)	Brier 2025-26 (model / baseline)	Predicted vs actual 2024-25	Predicted vs actual 2025-26
To score	0.0723 / 0.0762	0.0684 / 0.0725	8.4% vs 8.7%	7.9% vs 8.0%
To assist	0.0596 / 0.0640	0.0704 / 0.0750	6.8% vs 6.6%	7.5% vs 7.8%
Goal or assist	0.1112 / 0.1173	0.1182 / 0.1259	14.3% vs 14.4%	14.6% vs 15.0%

Market	Brier 2024-25 (model / baseline)	Brier 2025-26 (model / baseline)	Predicted vs actual 2024-25	Predicted vs actual 2025-26
1+ shots	0.1995 / 0.2076	0.2049 / 0.2144	46.9% vs 46.6%	46.1% vs 45.9%
1+ shots on target	0.1506 / 0.1559	0.1523 / 0.1593	22.9% vs 22.9%	22.0% vs 22.2%
2+ shots on target	0.0494 / 0.0513	0.0422 / 0.0434	5.9% vs 5.8%	5.1% vs 4.7%

- **Against the baseline:** the model won 21 of the 22 market × line × season rows. The 22nd, 3+ goals in 2025-26, tied at 0.0007 (hat-tricks are 0.1% of player-games).
- **Against rates only:** the team and opponent scaling helped almost everywhere, most for shots and shots on target. Scaling chosen: goals 0.75, assists 0.75, shots 1, shots on target 1.
- **Calibration:** "to score" is close from 0% to 60%; 30–40% is a little high (33.9% predicted against 29.1% actual).
- **Opponent effect:** Haaland's chance to score in 2025-26 ran from 63% (home to West Ham) to 38% (away at Arsenal), while his own rate alone gave 55–56% for both.
- **Fixed while testing:** (1) match_lambdas only had the 9 gameweeks a season your match model saved, so block 15.5a-ii fills the rest from the latest earlier run. (2) Assists per goal in the data moves between seasons (0.74 in 2024-25, 0.94 in 2025-26, 0.72 so far in 2026-27), so one pooled 0.828 gave 7.1% against 6.6% and 6.2% against 7.8%. It is now worked out for each week from that season's games so far, and the league level is always applied in full, with the scaling setting only controlling how much teams and opponents differ from it.

15.6 Discipline and defensive player markets. Built into the same script and table as 15.5 (`player_markets.py`, table `player_markets`), the same way: his 15.3 rate × his 15.2 minutes, scaled so a team's players add up to the team's expected total from 15.4, with the scaling setting chosen on 2024-25 and checked on 2025-26.

- **Fouls committed (1+, 2+, 3+):** scaled to his team's expected fouls, which already include the referee factor.
- **Fouled (1+, 2+, 3+):** scaled to the opponent's expected fouls, so a dribbler facing a foul-heavy side with a strict referee rises.
- **Tackles (1+, 2+, 3+):** scaled to his team's expected tackles.
- **To be carded (yellow or red):** yellow rate scaled to his team's expected yellows, plus his red chance.

- **Red card:** position-average red rate scaled to his team's expected reds. Too rare to judge week to week; judge on a season's calibration.
- **Keeper saves (1+ to 5+), keepers only:** the opponent's expected shots on target minus their expected goals, set to the league's actual saves per game so far (this season, shrunk towards earlier seasons), with the scaling setting chosen on 3+ saves.

Steps:

1. **Replace the script.** Put the new `player_markets.py` in the folder over the old one.
2. **Backtest.** Run `python player_markets.py backtest`. It now prints a scaling line for fouls, fouled, tackles, yellow, red and saves too, and saves 11 markets.
3. **Storage.** Run 15.6a: every market listed, with saves only for keepers.
4. **Test.** Run 15.6b. The model should beat the baseline in each row. Red cards and 1+ saves (over 90% of keepers make one) may only tie.
5. **Calibration and referee.** Run 15.6c (to be carded) and 15.6d (card and foul chances by referee strictness: strict referees should show higher predicted and actual percentages).
6. **Keeper check.** Run 15.6e: one keeper's chance of 3+ saves against every opponent; strong attacks should be near the top.
7. **Each week:** after `player_markets.py next`, run 15.6f for the likeliest to be carded, with fouls, fouled and tackles.

Calibration (added after the first 15.6 test, applies to 15.5 too). The first test showed chances that were too extreme: players given 0–10% to be carded were booked 8.6% of the time (5.9% predicted), those given 30–40% only 22.5%, and tackles 1+ ran 1.9 points high. Players have good and bad days beyond Poisson luck, and their rates carry noise. So each market now gets three numbers fitted on 2024-25: r (negative binomial spread), c (level) and g (how far to pull extreme players towards the middle, below 1 = pull in). A market keeps them only if they also improve 2025-26. All 10 markets kept them; the biggest pulls were fouls (g 0.69), tackles (0.78) and fouled (0.79), while goals barely moved (0.98).

Result on your data (adopted). Every one of the 54 market × line × season rows across 15.5 and 15.6 beats its baseline (the 3+ goals tie in 2025-26 apart).

Market	Brier 2024-25 (model / baseline)	Brier 2025-26 (model / baseline)	Predicted vs actual 2024-25	Predicted vs actual 2025-26
Fouls 1+	0.2210 / 0.2296	0.2229 / 0.2339	46.4% vs 46.6%	46.3% vs 46.7%
Fouled 1+	0.2174 / 0.2252	0.2167 / 0.2248	44.5% vs 44.8%	44.5% vs 44.2%
Tackles 1+	0.2059 / 0.2146	0.2002 / 0.2088	56.4% vs 56.3%	55.1% vs 55.1%

Market	Brier 2024-25 (model / baseline)	Brier 2025-26 (model / baseline)	Predicted vs actual 2024-25	Predicted vs actual 2025-26
To be carded	0.1167 / 0.1238	0.1101 / 0.1161	13.6% vs 13.8%	13.3% vs 12.9%
Red card	0.0047 / 0.0051	0.0039 / 0.0043	0.4% vs 0.5%	0.4% vs 0.4%
3+ saves	0.2346 / 0.2520	0.2428 / 0.2503	55.8% vs 54.7%	50.2% vs 52.8%
1+ shots (15.5)	0.1984 / 0.2076	0.2037 / 0.2144	46.3% vs 46.6%	45.6% vs 45.9%

- **To be carded calibration:** 0–10% band 6.8% predicted vs 8.0% actual, 10–20% 14.8% vs 14.1%, 20–30% 22.2% vs 22.4%.
- **Referee effect (fouls 2+):** lenient 17.2% predicted vs 16.8% actual, average 18.0% vs 18.0%, strict 18.7% vs 18.7%. Card chances don't vary by referee because 15.4 found the referee didn't help team yellows, and actual bookings by referee band (12.9 / 13.8 / 13.0%) agree. A weaker referee setting for yellows is still to be retested.
- **Keeper saves:** the opponent matters most here (3+ saves 0.2346 against 0.2520 league-only in 2024-25). Pickford's chance of 3+ saves in 2025-26 ran from 73% (away at Liverpool) to 37% (home to Burnley).
- **Watch:** assists in 2025-26 run a little low after calibration (7.1% vs 7.8%), though Brier improved; 2+ saves in 2025-26 run low (72.8% vs 75.9%).

15.6x Recent form and penalty takers. Checked against how bookmakers and analysts price player markets: they build on xG and shot volume (as 15.3 does), adjust for opponent and venue (15.4–15.5), and then for role and line-ups. Studies find scoring "form" is mostly noise (no hot hand in Premier League goal-scoring; StatsBomb found a pseudo-R^2 of about 0.002 for recent conversion), but role form (minutes, shot volume, who takes penalties) is real. Minutes (15.2) and team counts (15.4) already weight recent games; player rates did not, and penalty-taker changes only showed up once his xG caught up. Two additions, each tested with the usual rule:

- **Recent form in player rates:** this season's games weighted by a half-life (none, 365, 180, 90 or 45 days), chosen per stat on 2024-25 and kept only if 2025-26 also improves. His total minutes stay the same, so only the balance between recent and older games changes.
- **Penalty takers:** xG split into open-play xG (his own rate, penalties at 0.76 xG removed) plus his share of his team's penalties (from who took them before the cut-off, this season and last) × the league's penalties per match × 0.76. In 'next', FPL's listed penalty order sets the share (first choice 85%, second 10%, third 5%).

Steps:

1. **Penalty data.** Replace `player_data.py` and run `python player_data.py history`. It adds a `penalties` column to `pl_player_matches` (scored + missed).
2. **Form test.** Replace `player_rates.py` and run `python player_rates.py form`. It builds the rates five times (several minutes), prints each stat's Brier for every half-life in both seasons and the penalty test, and saves the winners to `player_rates_form.json`.
3. **Rebuild.** Run `python player_rates.py backtest` (its first line says which settings are on), then `python player_markets.py backtest`.
4. **Check.** Rerun 15.5c and 15.6b. Keep the changes only if no market's Brier gets worse in either season; to switch them off, delete `player_rates_form.json` and rerun step 3.

Result on your data (adopted). Recent form made no real difference: weighting to the last 45 days made shots, shots on target, fouls, fouled, tackles and yellows worse in both seasons (fouls 0.2040 to 0.2056 in 2024-25), and the longer half-lives the test picked (assists 365 days, tackles 365, red 180) differ from "all games equal" only beyond the fourth decimal place, a tie in practice. Penalty takers improved goals in both seasons (0.0707 to 0.0706, 0.0673 to 0.0672), with regular takers gaining up to about 20% (0.23 to 0.28 xG per 90). After rebuilding, player_markets was unchanged or better in every market: goals across all lines 0.0268 to 0.0267 in 2024-25, the rest identical. Finding for the write-up: scoring form was tested and adds nothing beyond xG and shot volume, which matches the published research; penalty roles add a little.

15.7 Team and match markets, and referee forecasts. A new script, `match_markets.py`, turns 15.4's expected counts into chances at every common line, for the home team, the away team and the match, into the table `match_markets`:

- **Corners:** team 1.5–9.5, match 6.5–14.5.
- **Fouls:** team 6.5–15.5, match 16.5–28.5 (the referee is already in 15.4's fouls).
- **Cards** (yellow + red, each counting 1): team 0.5–4.5, match 1.5–7.5.
- **Booking points** (10 per yellow, 25 per red, so a second-yellow red is 35): team 5.5–45.5, match 15.5–75.5.
- **Red card:** at least one, for each team and the match.
- **Most corners and most cards:** home, draw or away.

Team counts use a negative binomial around 15.4's expected count. Match totals are either the two teams combined, or one negative binomial with its own spread, which allows for whole-match effects (a strict referee or a heated game pushing both teams up together). Both spreads are chosen per stat on 2024-25 and kept only if 2025-26 agrees. The baseline is the same built around 15.4's league average with home advantage. Bookmakers settle cards differently (some count a red as 2, some cap booking points per player), so check a site's rules before comparing prices.

Referee forecasts. The view `v_referee_forecast` shows, for each upcoming match: the referee (or "referee TBC"), his foul factor, the expected fouls, cards and booking points, the chances of over 3.5 and 4.5 cards, over 20.5 and 22.5 fouls, over 35.5 booking points and a red card, plus his own per-game averages and the league's. Block 15.7f tests the forecasts against what each regular referee actually gave in 2024-25 and 2025-26.

Steps:

1. **Weaker referee settings.** Replace `team_counts.py` (the referee grid now also tries 60 and 100 matches, a weaker referee effect, so cards get another chance to use the referee). Run `python team_counts.py backtest` (about 10–15 minutes); note the referee setting it prints for fouls, yellows and reds.
2. **Match markets.** Put `match_markets.py` in the folder and run `python match_markets.py backtest`. It prints the spreads chosen and a Brier table, model against baseline, for every market in both seasons.
3. **Storage.** In pgAdmin, run 15.7a: every market and side with the same number of fixtures.
4. **Test.** Run 15.7b (all lines) and 15.7c (the common match lines one by one). The model should beat the baseline, and predicted should be close to actual.
5. **Calibration.** Run 15.7d (match cards over 3.5 by chance band).
6. **Referees.** Run 15.7f (predicted against actual cards and fouls for each regular referee), then create the view in 15.7e.
7. **Each week:** after `python player_data.py referees load`, run `python team_counts.py next`, then `python match_markets.py next`, then `SELECT * FROM v_referee_forecast`.

Result on your data (adopted). The 15.7 retest of 15.4 left fouls using the referee (now a weaker K 100) and confirmed no referee effect for yellows; a referee setting made reds slightly worse than the baseline in both seasons, so reds use none (league average, a tie). Match totals work best as the two teams combined with 15.4's own spreads.

Market (all lines)	Brier 2024-25 (model / baseline)	Brier 2025-26 (model / baseline)
Fouls	0.1673 / 0.1843	0.1711 / 0.1783
Most corners	0.1673 / 0.1901	0.1851 / 0.1937
Corners	0.1657 / 0.1761	0.1619 / 0.1675
Cards	0.1537 / 0.1574	0.1450 / 0.1471
Booking points	0.1581 / 0.1620	0.1501 / 0.1525
Most cards	0.2134 / 0.2195	0.2140 / 0.2186

Market (all lines)	Brier 2024-25 (model / baseline)	Brier 2025-26 (model / baseline)
Red card	0.0764 / 0.0763	0.0589 / 0.0589

- **Common lines:** over 3.5 match cards 59.3% predicted vs 60.5% actual (2024-25) and 55.7% vs 56.8%; over 20.5 fouls 61.5% vs 61.6% and 56.6% vs 56.8%; over 35.5 booking points 60.9% vs 62.2% and 57.5% vs 57.0%.
- **Calibration** (match cards over 3.5): 40–50% band 45.9% vs 47.6% actual, 50–60% 55.1% vs 57.0%, 60–70% 64.6% vs 63.2%, 70–80% 73.5% vs 74.1%.
- **Match corners only tie the baseline** (0.1847 vs 0.1850, 0.1826 vs 0.1822), though team corners and "most corners" win clearly: good teams win more corners and concede fewer, so the total barely changes between fixtures. Shown on the dashboard, flagged as no better than the league average. Home corners in 2024-25 ran high (53.4% vs 48.0%), the 2024-25 corners bias already seen in 15.4; 2025-26 is close (49.1% vs 48.2%).
- **Referees:** predicted fouls track each referee closely (for example R Jones 22.7 vs 23.3, C Kavanagh 21.4 vs 22.0). Actual cards per game ranged from 3.54 (C Pawson) to 4.62 (J Brooks) while predicted stayed 3.97–4.19, but with 24–59 games each that spread is about what chance alone produces (one referee's average over 40 games varies by roughly ±0.3 cards), which is why a referee factor didn't help cards. The forecast view still shows each referee's own averages beside the model.

15.8 Slip checker (optional, about 4–5 h). A bet builder's legs are linked: Haaland scoring and City winning tend to happen together, so multiplying the single chances gives the wrong answer. `slip_checker.py` simulates the fixture 20,000 times:

- Draw the score from your match model.
- Give each goal to a scorer by xG share and minutes, and each assist by xA share.
- Draw shots, corners, fouls and cards from their 15.4–15.6 averages.

The chance of a slip is the share of simulations where every leg lands. Type up to six legs on the dashboard's Slip sheet, run the script, Refresh All. It shows:

- the joint chance,
- the product of the single chances (to show how much the link matters),
- and, if you type a site's price, the chance that price implies (1 ÷ decimal odds).

Bet builders carry larger margins than single markets, so expect most priced slips to sit above your model's chance. One honest limit: corners and cards are drawn from their pre-match averages, so in the simulation they don't react to the score.

Each simulated match: the score from your model's expected goals (matched to the ensemble); each player starts, comes on or doesn't play (15.2); each goal goes to a player on the pitch in proportion to his scoring rate × minutes (3% own goals), assists the same

way; his shots on target = his goals + extra on target, so a scorer always has one; each team's fouls, tackles and cards (15.4, referee included) are shared among its players by their rates, fouled = the opponent's fouls; saves = the opponent's shots on target that weren't goals; corners per team. Player legs are worked out given the named players play (bookmakers void them otherwise), and the chance they all play is shown.

Steps:

1. **Test.** Put `slip_checker.py` in the folder and run `python slip_checker.py test` (about 5–10 minutes). For three standard bet builders on every 2024-25 and 2025-26 fixture (home win + their likeliest scorer; that scorer scores + 2 or more shots on target; over 2.5 goals + over 3.5 cards) it prints the actual hit rate, the simulated chance and the single chances multiplied, with Brier for both.
2. **Check.** In pgAdmin run 15.8a (the same table) and 15.8b (calibration of the simulated chance).
3. **Your slips.** Run `python slip_checker.py example`, open `slip.csv`, change it to next gameweek's fixtures, players and lines (one row per leg; the site's price on any one row), save, and run `python slip_checker.py`. It prints each leg's chance, the chance all legs land with fair odds, the product of singles, and your edge at the site's price; 15.8c shows the saved results.

Result on your data (adopted).

Bet builder	Season	Actual	Simulated	Singles multiplied	Brier (simulated / multiplied)
Home win + their likeliest scorer	2024-25	20.8%	21.8%	15.7%	0.1526 / 0.1550
	2025-26	18.5%	18.7%	12.8%	0.1415 / 0.1455
Scorer + 2 or more shots on target	2024-25	20.2%	18.2%	9.4%	0.1579 / 0.1689
	2025-26	15.1%	14.0%	6.3%	0.1212 / 0.1295
Over 2.5 goals + over 3.5 cards	2024-25	34.0%	33.5%	33.5%	0.2200 / 0.2200
	2025-26	32.3%	28.5%	28.4%	0.2158 / 0.2161

For linked legs the simulation lands within about 2 points of what happened, while multiplying the singles understates the slip by about a third (win + scorer) to more than half (scorer + shots on target). Goals + cards come out the same both ways, as expected, because cards don't react to the score in the simulation; that combo ran about 4 points low in 2025-26, from the cards side rather than the linking.

15.9 Testing summary. Every market was tested as it was built (15.2–15.8): predicted from data available before each gameweek, scored against what happened, model against baseline, chosen on 2024-25 and confirmed on 2025-26. 15.9 gathers it into one view, `v_market_tests` (script `30_upgrade15_summary_dashboard.sql`): every market, line and season with actual and predicted chance, Brier for the model and the baseline, and "beats baseline / tie / worse". The rule for the dashboard: a market is shown if it beats its baseline over all its lines in both seasons; single extreme lines (for example over 14.5 corners) can tie or lose without dropping the market. Red cards and match corners tie, so they are shown flagged "no better than league average".

Steps:

1. In pgAdmin run 15.9a (creates `v_market_tests`), then 15.9b (counts per group and season) and 15.9c (everything that tied or lost).
2. Fill in the Upgrade log row below from 15.9b.

15.10 Dashboard. Six new views feed five new sheets in Prem_Dashboard.xlsx, loaded over ODBC the same way as the earlier sheets (Data → Get Data → From Other Sources → From ODBC → your DSN → prem_sims → public → tick the view → Load To → Table):

Sheet	Reads	Shows
Player markets	`v_player_markets_next`	every player in next gameweek's fixtures: start chance, minutes, xG, xA and every player market and line, with slicers for fixture, team and position
Match markets	`v_match_markets_next`	each fixture: expected corners, fouls, cards and booking points, the common over/under lines, most corners / most cards, red card
Referees	`v_referee_forecast`, `v_referee_stats`	next gameweek's referees with their forecast for the game and their own per-game averages; the all-referee table with a season filter
Player vs opponent	`v_player_vs_opponent`, `v_player_list`	pick a player from a drop-down: his chance to score, assist, goal or assist, 1+ shots and 1+ shots on target against every other team, home and away, sorted, with a bar chart

Sheet	Reads	Shows
Slip	`slip_results`	the latest slips checked with slip_checker.py: chance, fair odds, site price and value
Market tests	`v_market_tests`, `v_market_calibration`	the 15.9 table and a predicted-vs-actual calibration chart for the main markets

Player vs opponent is an approximation: his chances in his real next fixture are scaled by how his team's expected goals against each opponent (from your model's ratings for that gameweek) compare with that fixture. Real fixtures use the exact numbers on the Player markets sheet.

Steps:

1. **Views.** In pgAdmin run blocks 15.10a–e. If 15.10c is slow, run `ANALYZE;` once and try again. The "next" views stay empty until your first weekly next run (GW6 or GW7).
2. **Sheets.** In Prem_Dashboard.xlsx add the five sheets, loading each view as a table at A5 (rows 1–4 for a title and notes). For Player vs opponent, load `v_player_vs_opponent` and `v_player_list` to a hidden sheet "PvO data" instead.
3. **Formatting.** Use the Claude Excel add-in prompts given with these steps for each sheet (slicers, colour scales, the drop-down, the FILTER formula and the charts).
4. **Refresh.** Data → Queries & Connections → each new query → Properties → tick "Refresh data when opening the file", so Refresh All updates every sheet.

Weekly, once built:

1. Do the normal run (results, main model, 15.5a insert).
2. Fill in referees_next.csv and run `python player_data.py referees load`.
3. Run `python player_data.py update`, then `python player_minutes.py next`, `python player_rates.py next`, `python team_counts.py next`, `python player_markets.py next` and `python match_markets.py next`.
4. Optional: fill in slip.csv and run `python slip_checker.py`.
5. Open the dashboard and Refresh All.

Upgrade 17 adds these steps to weekly.py.

15.9 Result. 126 market-lines tested per season (27 player, 96 team / match, 3 bet builders):

Season	Beat the baseline	Tie	Worse	Average Brier (model / baseline)
2024-25	117	3	6	player 0.1163 / 0.1219, team / match 0.1605 / 0.1703, bet builders 0.1768 / 0.1813
2025-26	110	4	12	player 0.1157 / 0.1203, team / match 0.1573 / 0.1620, bet builders 0.1595 / 0.1637

Every player line beats its baseline except 3+ goals in 2025-26 (a tie at 0.0007). The ties and losses are all team / match lines, and nearly all by the third or fourth decimal place: match corners at most lines (expected: the total barely changes between fixtures), red cards (league average, a tie), and tail lines such as over 65.5 booking points, 4.5+ cards for one team and 6.5–7.5 home fouls in 2025-26. Every market still beats its baseline across all its lines in both seasons, so all are kept; match corners and red cards are flagged on the dashboard as no better than the league average.

Upgrade log row: "15. Player and match markets — kept: 11 player markets (score 1–3+, assist, goal or assist, shots 1–3+, shots on target 1–3+, fouls, fouled, tackles 1–3+, carded, red, saves 1–5+), 7 team / match markets (corners, fouls, cards, booking points, red, most corners, most cards), referee forecasts and the bet builder slip checker. Market-lines beating the baseline: 117 of 126 in 2024-25, 110 of 126 in 2025-26. Average Brier, player 0.1163 vs 0.1219 and 0.1157 vs 0.1203; team / match 0.1605 vs 0.1703 and 0.1573 vs 0.1620; bet builders 0.1768 vs 0.1813 and 0.1595 vs 0.1637. Tested and dropped: recent-form weighting (no gain), referee factor for cards (referee spread is mostly chance) and reds. Flagged: match corners and red cards tie the league average."

Upgrade 16: take the database password out of the code

Why: your password sits in plain text on line 25 of dc_model.py, and every other script borrows it from there. Moving it to an environment variable first means Upgrade 17's automation and Upgrade 18's public repository are built on the safe setup. About 30 minutes.

Every script already reads `PREM_DB_URL` from the environment first and only falls back to `dc_model.DB_URL` if it isn't set, so nothing else needs changing.

1. **Set the variable** (PowerShell, once): `setx PREM_DB_URL "postgresql+psycopg2://postgres:YOUR_PASSWORD@localhost:5432/prem_sims"` with your real password, `%40` for any `@`. It should say SUCCESS.
2. **Restart VS Code completely** (close every window), so its terminal sees the new variable. Check with `python -c "import os; print('set' if os.environ.get('PREM_DB_URL') else 'NOT set')"`.

3. **Test a connection** before removing anything: `python -c "import player_minutes as p; from sqlalchemy import text; print(p.engine().connect().execute(text('SELECT COUNT(*) FROM matches')).scalar())"` prints your match count.
4. **Take the password out:** in dc_model.py set line 26 back to `YOUR_PASSWORD`, save, and run the step 3 test again. It must still print the count.
5. **Find old copies:** in the Prem Project folder, run the search in these steps to list any file that still holds a real connection string (older DC upgrade folders, backups). Replace the password with `YOUR_PASSWORD` or delete those copies.
6. **Excel:** in Prem_Dashboard.xlsx and the model, Data → Queries & Connections → each connection → Properties → Definition: if the connection string contains `PWD=`, remove that part and untick Save password. The ODBC DSN keeps its own password on your PC only, which is fine.
7. **Optional, recommended:** the old password has sat in a file synced to OneDrive, so change it: in pgAdmin `ALTER ROLE postgres PASSWORD 'new password';`, then rerun step 1 with the new one, restart VS Code, and update the password in pgAdmin's saved connection and the ODBC DSN.

Result: the scripts run as before, and the .py files are safe to share or upload in Upgrade 18.

GW6 weekly run: fixes and new dashboard views

The first real run of weekly.py (16 Oct 2026, "2026-27 after GW6") worked after six fixes. The GW7 markets, the Player sheet and the expanded H2H were built on that run.

File	What was fixed
weekly.py	Model file name (`prem simulation 26-27 EXPORT.xlsx`). Understat sends its data gzip-compressed; it is now unpacked. Builds xg.csv and odds.csv itself (the old make_weekly_csvs.py takes its arguments in a different order). Reads availability.csv with its byte-order mark and matches team names as block 14.5d does. Clears its own arguments before running market_model, which read them and failed. Checks team names and the label before writing anything.

File	What was fixed
player_minutes.py	A doubtful player's start chance is capped at his FPL chance x 0.97. Before, scaling the team back to 11 starters handed the cut back to him (Isak at 25% fit showed a 94% start). Backtests are unchanged: they have no FPL flags.
slip_checker.py	Works on upcoming games (blank actual counts crashed it). `slip_results` gains fixture, legs_detail and fair_odds.

Known limit: the slip checker simulates cards and fouls separately, so a fouls + cards builder comes out equal to the product of its singles. The real joint chance is a little higher.

New views (script 30). 15.10f `v_player_card`: one row per player, market and line (chance if he plays, expected count), with his fixture, referee, start and play chance, and both teams' expected goals. 15.10g `v_fixture_card`: every team and match market for each team's next fixture, from that team's side. It covers result (main model and model + market blend), team and match goals, both teams to score, clean sheet, corners, fouls, cards, booking points, red card, most corners and most cards, and season chances. The goals lines come from the tested expected goals but are not yet backtested on their own.

Dashboard. New Player sheet: pick a player for his summary, his markets and his team's match. H2H rows 44-120: team goals, clean sheet, win to nil and draw no bet for any pair; corners, fouls, cards and booking points ladders, the referee and the blended result for next-gameweek fixtures (home team first). Slip sheet from `slip_results`. Data sheets: Data Players (`v_player_card`) and Data fixture card (`v_fixture_card`).

Result. Run "2026-27 after GW6": 20 ratings, 20,000 sims, 380 predictions per model, 20 availability rows and 10 GW7 fixtures in every market table. Man City 57.3% title, Arsenal 28.3%.

Upgrade 17: automate the weekly pipeline

Why: GW6 took about two hours by hand: weekly.py part1 and part2, then a pgAdmin insert, the referees, six scripts and a dozen checks. Upgrade 17 adds `part3`, which does the Upgrade 15 steps, and `check`, which prints every check as OK or CHECK. Excel keeps only what needs Excel: typing scores, pasting availability, refreshing xG, F9 and Refresh All. A week should take about 15 minutes of your time.

17.1 One-time setup

1. Replace weekly.py in the DC upgrade (6) folder with the Upgrade 17 version. Keep player_data.py, player_minutes.py, player_rates.py, team_counts.py, player_markets.py and match_markets.py in the same folder.
2. Run script 30 blocks 15.10f and 15.10g once, if not done: the checks read both views.
3. Test the checks on last week's run: `python weekly.py check "2026-27 after GW6"`. Every line should say OK.

17.2 Each week (the label names the gameweek just played)

1. Excel: type the scores on Results, Ctrl+S, close.
2. VS Code: `python weekly.py part1 "2026-27 after GW7"`.
3. Excel: paste `exports\<label>\availability.csv` into Availability (paste values), set NextGW, refresh the xG table, F9, Ctrl+S, close.
4. VS Code: `python weekly.py part2 "2026-27 after GW7"`. The ensemble step takes about 3 minutes.
5. VS Code: `python weekly.py part3 "2026-27 after GW7"`. The first time, it writes referees_next.csv and stops. Fill in the names from premierleague.com "Match officials for Matchweek N" (full names are fine), save, and run part3 again. It then loads the referees, runs 15.5a for this run, updates the player data, runs the five next scripts and prints the checks.
6. Dashboard: add the label at the bottom of Lists column A, pick it in H2H C6, then Refresh All.

Checks printed by part3 and check

Check	Expect
part1: played vs with xG	equal
part1: availability snapshot	20 teams
part2: ratings and sims	20 and 20,000
part2: dixon_coles, market, ens_mkt_50, pi_ratings, dc_xg	380 each
15.5a expected goals	one per next-gameweek fixture
referees	every fixture has one, every one has history
player_minutes	11 expected starters per team; nobody starting above his FPL chance
team_counts / match_markets	21 / 100 rows per fixture
player_markets	every fixture

Check	Expect
v_fixture_card / v_player_card	2 teams per fixture / 300+ players

Safety built in: part2 refuses a label that already exists (Troubleshooting F1). part3 refuses a label part2 has not loaded. Every part3 step replaces its own rows for the run, so rerunning after a fix is safe. A script that fails stops part3 and shows its error. `--skip-referees` carries on without referees (factor 1, "referee TBC") when appointments are not out yet.

If something fails: fix the step named in the error (or run it by hand), then run part3 again and send the CHECK lines. Test on GW7, then add the Upgrade log row.

Upgrade 18: package it as a portfolio project

Why: the work only counts for a job application if someone can see it in two minutes. Upgrade 18 puts the code, SQL, results and screenshots in one public GitHub repository with a README, without your password or any data you aren't allowed to share. About 4-6 hours, most of it the README and the write-up.

18.1 Make a clean copy (30 min). Never publish your working folder: it holds exports, cached data and personal paths.

1. Create `C:\Users\<you>\OneDrive\Prem Project\github\premier-league-forecasting` with the folders below.
2. Copy files in. Copy only what the table lists.

Folder	What goes in
`python/`	weekly.py, dc_model.py, benchmarks.py, market_model.py, dynamic_sims.py, fpl_availability.py, make_weekly_csvs.py, player_data.py, player_minutes.py, player_rates.py, team_counts.py, player_markets.py, match_markets.py, slip_checker.py, plus requirements.txt
`sql/`	every numbered script, 01 to 30, unchanged
`images/`	8-10 dashboard screenshots (18.4)
`docs/`	the two upgrade guides and the weekly guide exported as PDF, the Upgrade log as a PDF or CSV, and the write-up (18.5)

Folder	What goes in
`excel/`	optional: the model workbook only, with the data sheets cleared; leave the dashboard out (it holds FPL-Core-Insights player data)

3. requirements.txt, one per line: `pandas`, `numpy`, `scipy`, `sqlalchemy`, `psycopg2-binary`, `openpyxl`.
4. Check for secrets in PowerShell, inside the new folder: `Select-String -Path python\*.py -Pattern "postgresql"`. The only hit should be dc_model.py's `postgres:YOUR_PASSWORD@localhost`. Then `Select-String -Path python\*.py,sql\*.sql -Pattern "<your real password>"`: it must find nothing.
5. Optional privacy tidy: weekly.py's MODEL_PATH and EXPORT_ROOT show your Windows user name. Replace `<you>` with `YOUR_NAME` in the published copy only.

.gitignore (a file named `.gitignore` in the top folder, so these never get uploaded by mistake):

```
exports/
player_data/
player_data_cache/
snapshots/
__pycache__/
*.backup
*.csv
!docs/*.csv
referees_next.csv
.env
```

18.2 Publish with GitHub Desktop (30 min).

1. Make a free account at github.com and install GitHub Desktop.
2. In GitHub Desktop: File, Add local repository, choose the premier-league-forecasting folder, then "create a repository" when it offers. Name `premier-league-forecasting`, Git ignore None (yours is already there).
3. Read the list of changed files on the left. Anything from exports, player_data or a password means stop and fix the copy.
4. Summary `First version: model, SQL pipeline, markets, dashboard`, then Commit to main.
5. Publish repository, untick "Keep this code private", Publish.
6. Open the repository's page in a private browser window, check it shows the README and the folders, and copy the link for your CV and LinkedIn.

18.3 README (2 hours). This is the page people read. Keep it to about one screen of text plus images, in this order:

1. **One-line summary:** "A Premier League forecasting system: Excel Monte Carlo model, PostgreSQL pipeline, Python models for match, player and team betting markets, and a live Excel dashboard."
2. **Headline results (3-5 bullets):** league-position error 3.94 places pre-season to 1.94 by GW22 and 1.15 by GW34 (2025-26); match predictions within 3-4% of bookmaker accuracy (RPS 0.209-0.212 vs 0.200-0.206); 117 and 110 of 126 player and match market lines beat their baseline out of sample (2024-25, 2025-26); a value tracker testing flagged prices against the closing odds (add its result once it has 100+ flags).
3. **Two screenshots:** title race and the H2H sheet.
4. **How it works:** a short pipeline diagram in text: Excel model, then weekly.py part1/part2/part3, then PostgreSQL views, then Excel dashboard via ODBC and Power Query.
5. **What I tested and kept or dropped:** a small table from the Upgrade log, including the things that didn't help (recent-form weighting for goals, the referee factor for cards). Honest negatives make it credible.
6. **Data sources and licences:** football-data.co.uk, Understat, the official FPL API, FPL-Core-Insights (used locally, not redistributed: no licence file).
7. **Use of AI:** "Built with Claude as a development partner for Python and SQL; every model change was adopted only after out-of-sample tests, and the automated checks caught errors in AI-written code (for example the doubtful-player start fix in October 2026)."
8. **How to run:** install requirements, set `PREM_DB_URL`, run the SQL scripts in order, then the weekly commands from the weekly guide.

18.4 Screenshots (30 min). Win+Shift+S, save as PNG in `images/` with plain names: `title_race.png`, `model_comparison.png`, `calibration.png`, `h2h_value.png`, `player_card.png`, `hit_rates.png`, `market_tests.png`, `value_tracker.png` (once settled), `referees.png`. Refresh the dashboard first so every sheet shows the same run.

18.5 Write-up (2 hours). 1,200-1,500 words in `docs/`: the question, the data, the model and its upgrades, how each was tested (out-of-sample, both seasons), results against bookmakers and Opta, what didn't work, what you'd do next. Link it from the README.

18.6 CV and LinkedIn. Add the repository link to the project line on your CV. Post a short LinkedIn update with one screenshot and two results.

18.7 Keep it current. Once a month, copy the updated scripts and a fresh screenshot into the github folder, then Commit and Push in GitHub Desktop. At season end, add a "2026-27 review" section to the README with the final position error, the value tracker result and the Upgrade log.

Upgrade log row: "18. Published: public GitHub repository with code, SQL, README, write-up and screenshots; no password or licensed data included."